Code Generators and Compilers

In metaprogramming, you write code that writes code. But isn't that what code generators and compilers do? For example, you can write annotated Java code and then use a code generator to output XML configuration files. In a broad sense, this XML generation is an example of metaprogramming. In fact, many people think about code generation when the "M" word comes up.

This particular brand of metaprogramming implies that you use a program to generate or otherwise manipulate a second, distinct program—and then you run the second program. After you run the code generator, you can actually read the generated code and (if you want to test your tolerance for pain) even modify it by hand before you finally run it. This is also what happens under the hood with C++ templates: the compiler turns your templates into a regular C++ program before compiling them, and then you run the compiled program.

In this book, I'll stick to a different meaning of *metaprogramming*, focusing on code that manipulates itself at runtime. You can think of this as *dynamic* metaprogramming to distinguish it from the *static* metaprogramming of code generators and compilers. While you can do some amount of dynamic metaprogramming in many languages (for example, by using bytecode manipulation in Java), only a few languages allow you do to it seamlessly and elegantly—and Ruby is one of them.

A program written in C spans two different worlds: compile time, where you have language constructs such as variables and functions, and runtime, where you just have a bunch of machine code. Because most information from compile time is lost at runtime, C doesn't support metaprogramming or introspection. In C++, some language constructs do survive compilation, and that's why you can ask a C++ object for its class. In Java, the distinction between compile time and runtime is even fuzzier. You have enough introspection at your disposal to list the methods of a class or climb up a chain of superclasses.

Ruby is a very metaprogramming-friendly language. It has no compile time at all, and most constructs in a Ruby program are available at runtime. You don't come up against a brick wall dividing the code that you're writing from the code that your computer executes when you run the program. There is just one world.

In this one world, metaprogramming is everywhere. Ruby metaprogramming isn't an obscure art reserved for gurus, and it's not a bolt-on power feature that's useful only for building something as sophisticated as Active Record. If you want to take the path to advanced Ruby coding, you'll find metaprogramming at every step. Even if you're happy with the amount of Ruby you already know and use, you're still likely to stumble on metaprogramming in